

Part 2

Building knowledge graphs from structured data sources

This part of the book addresses the complex but essential process of constructing KGs from disparate structured data sources—a fundamental step before enriching them with unstructured information and combining them with large language models (LLMs). Organizations maintain vast repositories of data, each with its own schema, structure, and storage format. The challenge is harmonizing this data into a coherent KG while preserving its semantic meaning and relationships. We'll guide you through this process, demonstrating how to transform diverse structured data sources into a unified knowledge representation.

A key theme is the importance of data quality and validation, because the quality of downstream applications depends on the reliability of the underlying knowledge representation. You'll learn how to verify data integrity, ensure accurate entity matching, and validate the semantic correctness of KGs.

Chapter 3 presents a healthcare example, constructing a KG that helps clinicians diagnose rare diseases based on patient symptoms. It introduces fundamental concepts like semantic integration through ontologies, compares KG technologies, and provides hands-on implementation guidance.

Chapter 4 explores the progression from simple networks to comprehensive multisource integration across biomedical applications. It demonstrates advanced analysis methodologies, including community detection algorithms

and domain-specific metrics, and it introduces the integration of large language models for result interpretation and decision support.

The examples in these chapters demonstrate the technical aspects of KG construction and also illustrate how these principles can be applied to solve real-world problems in any domain.

Create your first knowledge graph from ontologies

This chapter covers

- Selecting the best KG technology based on use cases
- Constructing a KG to support clinicians' activities
- Performing analysis and ontology-based reasoning on top of a KG

KG construction is complex due to the need to extract and integrate information from data sources that differ in format (XML, CSV, JSON), storage technology (relational or document-oriented), information syntax (e.g., 2022-08-09 or 9 August 2022), and especially the meaning of the data. In healthcare, for instance, varied expressions that identify the same concept (type 2 diabetes versus ketosis-resistant diabetes), identical acronyms that define distinct concepts (PE as physical examination or pulmonary embolism), and information granularity (necrosis or lobular necrosis) are obstacles to data integration.

When constructing a KG, we aim for a unified, well-grounded, and meaningful representation of data from various sources, where individual pieces of information are integrated into a coherent view. Issues related to the meaning of data can be

addressed using semantic integration. A common strategy is to adopt one or more ontologies as a reference schema and vocabulary for incoming data. An *ontology* lets you model data using a standard vocabulary that includes elements such as formal names, properties, categories, and relationships between entities described within the data.

The ontology acts as an intermediary between semantically heterogeneous information. A *mapping* bridges a data source's local schema to the ontology's reference schema. We can map each data element to concepts expressed by the ontology; these annotations bring together data elements from different origins.

This chapter offers guidelines for building a KG using a reference ontology, focusing on helping clinicians identify rare diseases. We highlight data understanding and preparation, ingesting and processing the Human Phenotype Ontology (HPO; <https://hpo.jax.org/app/>) and an HPO-annotated dataset. The HPO source provides information about the connections between diseases and their associated phenotypic abnormalities. These abnormalities represent observable physical or biochemical characteristics that deviate from typical human traits and may result from genetic mutations, environmental influences, or a combination of both. We also explore the differences among KG technologies and offer a blueprint for selecting the most suitable option. Finally, we outline a set of analyses, including ontology-based reasoning, to support clinicians in diagnosing rare diseases.

Figure 3.1 provides a mental model for this chapter. The center shows the steps to create a KG in our example context, and the bottom illustrates an abstract pipeline

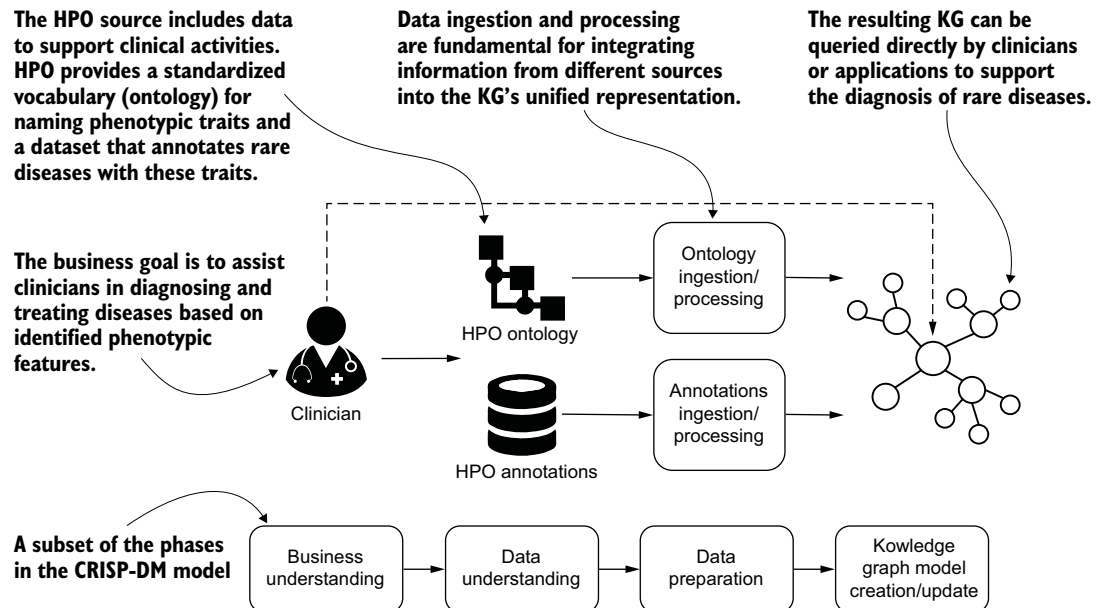


Figure 3.1 Mental model of the KG construction process as a specification of the CRISP-DM model, from understanding the business goal to defining the KG queries that support clinicians' activities

for constructing a KG that can be used in different scenarios. The components of this pipeline come from a version of the CRISP-DM model introduced in chapter 2 (and shown again in figure 3.2).

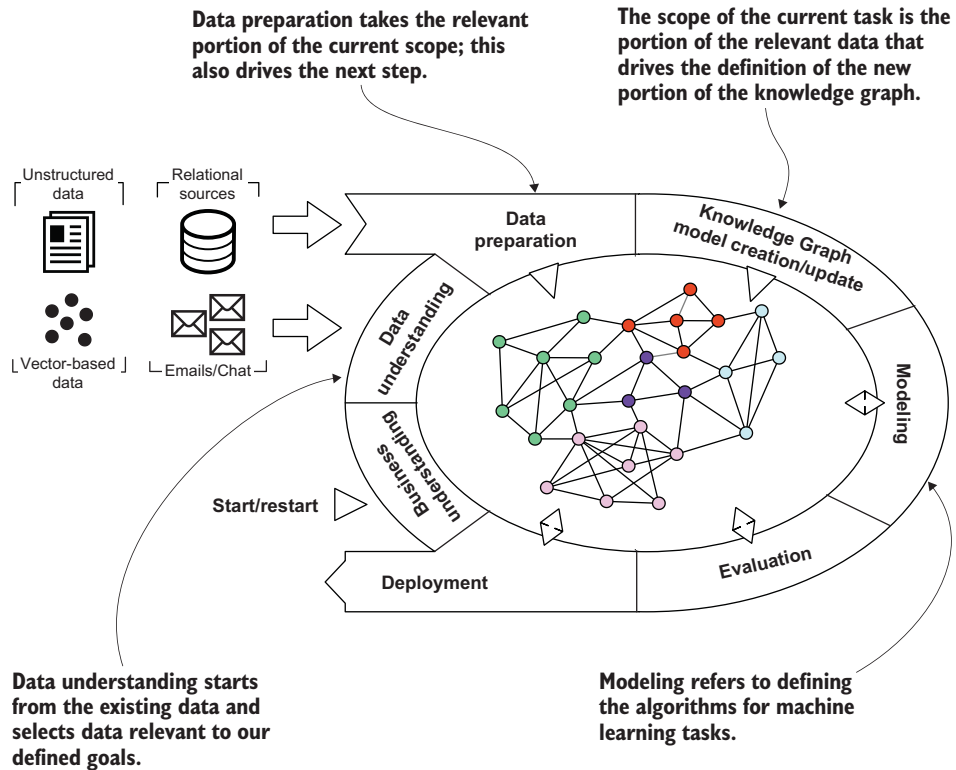


Figure 3.2 The CRISP-DM model adapted to KGs. A subset of these components, including business understanding, data understanding, data preparation, and KG model creation/update, are key phases described in this chapter.

3.1 Knowledge graph building: Warmup

Before creating a KG, we'll analyze the problem we want to solve, build an overview of the application domain, and scout for data.

3.1.1 Business and domain understanding

The target persona of our KG is the clinician: a healthcare professional who diagnoses and treats diseases. One of the clinician's most complex activities is correctly identifying a disease based on symptoms (phenotypic traits), particularly in the case of rare syndromes (see figure 3.3).

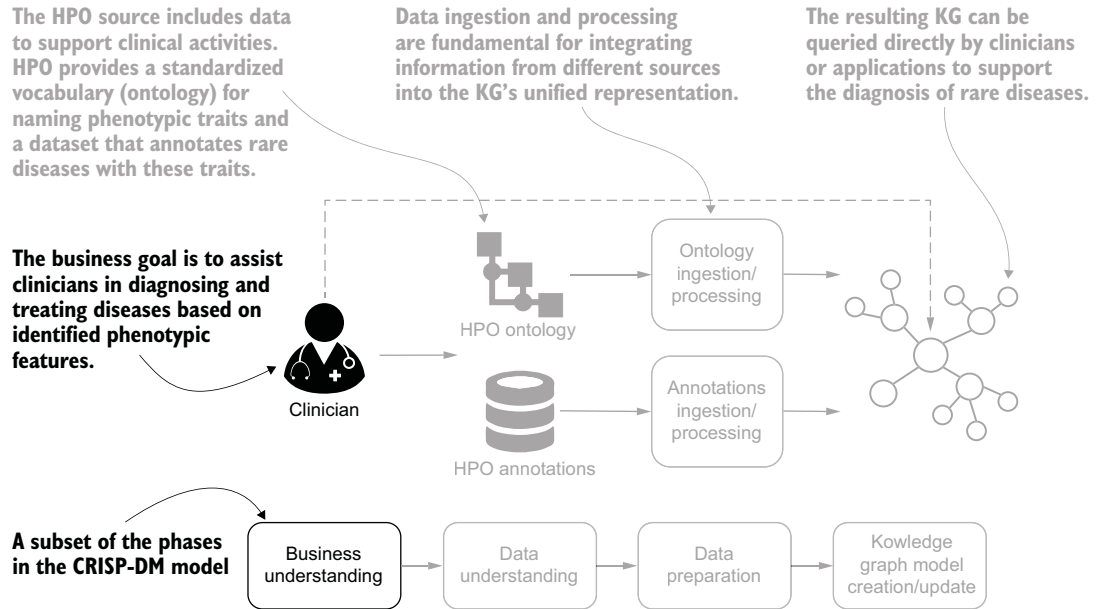


Figure 3.3 Understanding the business domain for creating a KG that supports clinicians' activities. This phase is not strictly related to the technical aspects, but it is fundamental for the next steps.

In addition to prescribing specific tests to reach a diagnosis, the clinician can use a structured knowledge base of available information. It should have these two features:

- *A contextual description of the phenotype domain*—For instance, phenotypic anomalies related to the same organs or systems should be explicitly connected.
- *Data describing the relationship between phenotypic anomalies and diseases*. This information must be tracked so clinicians can access the sources of the connections.

We want to build a KG that incorporates these features. To better understand the application domain, here are some definitions.

DEFINITION The *phenotype* of an individual with a disease can be said to be the sum of all of the phenotypic features manifested by that individual [1].

DEFINITION A *disease* is an entity characterized by (1) a set of causes for a specific condition, (2) a time course, (3) a group of phenotypic features, and (4) characteristic response to a particular treatment.

For example, the common cold is characterized by distinct phenotypic features, including fever and fatigue. The time course ranges from a couple of days to a week, and treatments such as aspirin can support healing.

But the clinician's work also involves gray areas. For instance, diabetes mellitus can be classified as a disease or as a phenotypic characteristic of other rare syndromes (see

figure 3.4). We'll work on this use case as an example of how we can help clinicians handle this kind of uncertainty.

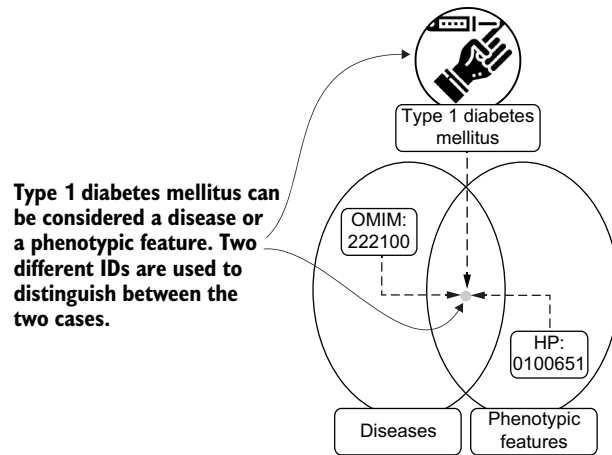


Figure 3.4 Type 1 diabetes mellitus can be considered either a disease or a phenotypic feature. Based on the context, two different IDs can be adopted.

3.1.2 Data understanding

Our data source is the Human Phenotype Ontology (HPO) repository. It provides two sets of information for our example (figure 3.5). The first, in an RDF/XML file called

The HPO source includes data to support clinical activities. HPO provides a standardized vocabulary (ontology) for naming phenotypic traits and a dataset that annotates rare diseases with these traits.

Data ingestion and processing are fundamental for integrating information from different sources into the KG's unified representation.

The resulting KG can be queried directly by clinicians or applications to support the diagnosis of rare diseases.

The business goal is to assist clinicians in diagnosing and treating diseases based on identified phenotypic features.

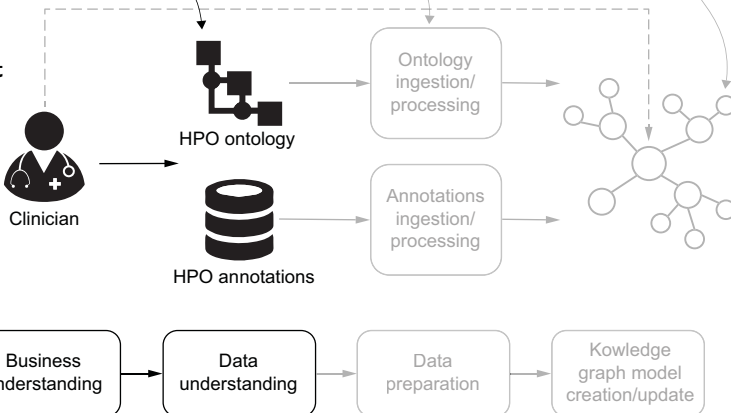


Figure 3.5 Understanding the data to support clinicians' activities. This explorative phase gets the key information needed to construct the KG.

hpo.owl (<http://purl.obolibrary.org/obo/hp.owl>), is an ontology that contains standardized information on phenotypic anomalies. This kind of standardization allows interoperability and lets us integrate data from multiple sources. Listing 3.1 shows part of the hpo.owl file related to Type I diabetes mellitus; the data is serialized to Turtle (Terse RDF Triple Language) for better readability.

Listing 3.1 Type I diabetes mellitus details in hpo.owl

```

obo:HP_0100651 a owl:Class ;
  rdfs:label "Type I diabetes mellitus" ^^xsd:string ;
  obo:IAO_0000115 "A chronic condition in which the pancreas produces
    little or no insulin..." ^^xsd:string ;
  oboInOwl:created_by "doelkens" ^^xsd:string ;
  oboInOwl:creation_date "2010-12-29T06:37:55Z" ^^xsd:string ;
  oboInOwl:hasDbXref "MSH:D003922" ^^xsd:string,
    "SNOMEDCT_US:46635009" ^^xsd:string,
    "UMLS:C0011854" ^^xsd:string ;
  oboInOwl:hasExactSynonym "Diabetes mellitus Type I" ^^xsd:string,
    "Juvenile diabetes mellitus" ^^xsd:string,
    "Type 1 diabetes",
    "Type I diabetes";
  oboInOwl:hasRelatedSynonym "Insulin-dependent diabetes
    mellitus" ^^xsd:string ;
  oboInOwl:id "HP:0100651" ^^xsd:string ;
  rdfs:comment "The onset of type 1 diabetes is typically during
    adolescence..." ^^xsd:string ;
  rdfs:subClassOf obo:HP_0000819 .

```

Defines Type I diabetes mellitus, identified by URI obo:HP_0100651, as an ontology class

Describes the disease in natural language

Shows metadata related to the author ("doelkens") of this entry

IDs of external data sources that refer to this form of diabetes

Defines Type I diabetes mellitus as a subclass of the phenotypic feature identified by the obo:HP_0000819 URI, which corresponds to diabetes mellitus

Reading an OWL file can be challenging. We can use the `rdflib` Python library to explore this file as a collection of triples, each of which includes a subject, a predicate, and an object, as shown in listing 3.2.

Listing 3.2 Processing an OWL file using the `rdflib` Python library

```

from rdflib import Graph, URIRef
g = Graph()
g.parse("hp.owl", format="xml")

g.bind("obo", "http://purl.obolibrary.org/obo/")
g.bind("rdf", "http://www.w3.org/1999/02/22-rdf-syntax-ns#")
g.bind("rdfs", "http://www.w3.org/2000/01/rdf-schema#")
g.bind("xsd", "http://www.w3.org/2001/XMLSchema#")

subject_uri = URIRef("http://purl.obolibrary.org/obo/HP_0100651")
filtered_statements = g.triples((subject_uri, None, None))
for subject, predicate, obj in filtered_statements:

```

```
print(
    f"({g.qname(subject)}, {g.qname(predicate)}, "
    f"{g.qname(obj)} if isinstance(obj, URIRef) else obj})"
)
print()
```

The output from this script is as follows (long strings have been truncated for clarity).

Listing 3.3 Sample OWL file shown as a set of triples

```
(obo:HP_0410050, rdf:type, owl:Class)
(ofo:HP_0410050, owl:equivalentClass, N25507ac984704bd78a0effd951947a7f)
(ofo:HP_0410050, rdfs:subClassOf, obo:HP_0011013)
(ofo:HP_0410050, obo:IAO_0000115, A decrease in the level of...)
(ofo:HP_0410050, dc:date, 2018-01-27T00:26:24+00:00)
(ofo:HP_0410050, dcterms:creator, ns1:0000-0001-5208-3432)
(ofo:HP_0410050, oboInOwl:hasExactSynonym, Decreased level of 1,5-AG...)
(ofo:HP_0410050, oboInOwl:hasExactSynonym, Decreased level of 1,5-anhydro...)
(ofo:HP_0410050, rdfs:label, Decreased level of 1,5 anhydroglucitol in serum)
```

The second set of information from HPO repository, contained in the tab-separated-values (TSV) file `phenotype.hpoa`, collects recognized, discovered, and annotated phenotypic features associated with different diseases, including rare syndromes. These annotations include modifiers that clarify the age of onset and the frequency of each feature related to the illness. The following listing shows a sample of this annotation file.

Listing 3.4 Sample of the `phenotype.hpoa` file

database_id	disease_name	qualifier	hpo_id	reference	evidence
onset	frequency	sex	modifier	aspect	biocuration
OMIM:222100	Diabetes mellitus, insulin-dependent-1				
HP:0410050	PMID:9357814;PMID:17659063;PMID:16731998				
PCS	30/30	P			
HPO:NicoleVasilevsky	[2018-02-23];HPO:NicoleVasilevsky				
OMIM:222100	Diabetes mellitus, insulin-dependent-1				
HP:0000103	OMIM:222100				
IEA		P		HPO:iea	[2009-02-17]

This file includes the following fields:

- `database_id` (OMIM:222100)—Disease identifier from ontologies such as Online Mendelian Inheritance in Man (OMIM) and Orphanet.

- `disease_name` (Diabetes mellitus, insulin-dependent-1)—Disease name from the related ontology.
- `hpo_id` (HP:0410050)—HPO identifier of the related phenotypic abnormality.
- `reference` (PMID:9357814;PMID:17659063.PMID:16731998)—Source of information used for the annotation. This may be from an article indicated with the related PubMed ID (PMID).
- `evidence` (PCS)—Level of evidence supporting the annotation. PCS stands for published clinical study.
- `frequency` (30/30)—A count of patients affected within a group of people with a common statistical characteristic. 30/30 indicates that 30 of the 30 patients with the specified disease were found to have the phenotypic abnormality referred to by the HPO term.
- `aspect` (p)—Phenotypic aspect. P means phenotypic abnormality.
- `biocuration` (HPO:NicoleVasilevsky[2018-02-23];HPO:NicoleVasilevsky[2018-03-02])—research center or user making the annotation and the date on which the annotation was made.

For further details, see <https://mng.bz/EwAo>.

3.2 **Understanding knowledge graph technologies**

Now that we understand the data, the next phase involves ingesting and processing it from the available sources. But first, we will examine the different KG technologies so we can make an informed decision for our use case.

Two of the most popular approaches for creating KGs are the Resource Description Framework (RDF) and Labeled Property Graph (LPG). RDF is a standard framework, defined and regulated by the World Wide Web Consortium (W3C), for data exchange on the web. With RDF, each statement is composed of three elements: subject, predicate, and object (a triple). The subject is a node (vertex) in the graph, the predicate represents a relationship (edge), and the object is another node. This framework models a KG as a collection of statements, and we can use web technologies to represent, store, and exchange information. RDF is particularly suitable for creating ontologies that describe a specific domain of knowledge.

LPG provides a fast, query-based traversal of graph data and path analysis features. Efficiency of data storage and access is guaranteed by the structured information in the form of key-value pairs associated with nodes and relationships in the graph.

In RDF, relationships (triples) are defined globally, so metadata applied to a predicate affects all instances of that relationship throughout the graph. To address this limitation, RDF supports, for instance, named graphs, which let us treat groups of triples as a single entity and provide context-specific information. In contrast, LPG supports unique edges between nodes, allowing metadata and properties to be attached to individual relationships. This is a flexible model for representing edge-specific

information. The RDF-DEV Community Group is working on an RDF* (“RDF-star”) specification that lets users add properties to edges, reconciling the RDF and LPG technologies.

LPG can’t express the advanced semantics of RDF. To address this issue, vendors such as Neo4j provide tools that can reduce the gap between RDF and LPG. The Neo-*semantics* plugin lets us use RDF and its vocabularies (OWL, RDFS, SKOS, and others) in Neo4j to run basic inference. Other vendors, such as Amazon Neptune, use alternative strategies that let us execute Cypher queries (the query language of LPG graphs) on RDF data. The next section presents the limitations and opportunities of adopting RDF and LPG for our example use case.

3.2.1 *RDF or LPG? A goal-driven discussion*

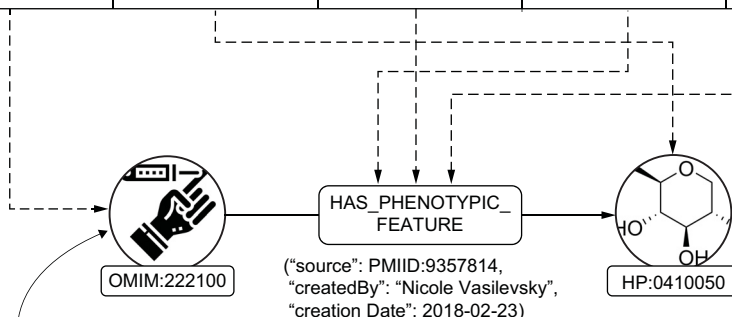
To select the best technology for building a KG, we need a better understanding of the available information (in our case, the HPO ontology and the annotations data) and a clear goal. We mentioned that RDF is particularly suitable for creating ontologies; this is why the HPO ontology is serialized using RDF. The file extension of the HPO file is .owl. OWL stands for Web Ontology Language, and its primary goal is to enrich the semantic information available in RDF to support expressive class definitions and property definitions. OWL ontologies are widely used, and many LLMs, including GPT and Claude, have been trained on them, making it easier for these models to interpret and reason over OWL-based data.

The clinicians in our use case don’t care how the knowledge is modeled: they are interested in an unambiguous representation of phenotypic features, possibly in a hierarchical structure. The core information in the annotated data often comes from the scientific literature and consists of cases in which a specific phenotypic feature is identified with a disease. For example, the entry that shows a connection between “Diabetes Mellitus, Insulin-dependent-1” (OMIM:222100) and “Decreased level of 1,5 anhydroglucitol in serum” (HP:0410050) was published in a clinical study entitled “A kinetic mass balance model for 1,5-anhydroglucitol: applications to monitoring of glycemic control” [3] (PMID: 9357814), created by Nicole Vasilevsky in February 2018. The best way to model it is to incorporate the details into a relationship between a disease and a phenotypic feature. Modeling data this way lets us create multiple relationships, each potentially representing a specific annotation characterized by a provenance and date.

Figure 3.6 illustrates how we can convert data from annotations in a table structure into an edge in the KG. The disease and the phenotypic feature are represented as nodes, and information about the annotation author, creation date, and source is specified as properties of the edge (`HAS_PHENOTYPIC_FEATURE` in the figure).

A simplified example of a row in the HPO annotations file.
This entry describes the association between a disease
(OMIM:222100) and a related phenotypic feature (HP:0410050).

Disease	Phenotypic feature	Author	Date	Source
OMIM:222100	HP:0410050	Nicole Vasilevsky	2018-02-23	PMID:9357814



Each entry value is used to define a property of a node or relationship in the knowledge graph. In this case, the annotation metadata ('Author,' 'Date,' and 'Source') specifies the properties of the edge connecting the disease (OMIM:222100) to a related phenotypic feature (HP:0410050).

Figure 3.6 Data transformation from a table row to a KG edge. Information in the table is adapted to define the properties of nodes and edges in the KG.

Exercise

See if you can select the best technology to support clinicians' activities for the example use case. Here's a reminder of the primary requirements:

- The clinician's goal is to use available data to make informed decisions when diagnosing diseases, especially rare pathologies.
- Clinicians are not interested in a knowledge base that represents the entire clinical domain. They want to see cases in which anomalous phenotypic features (or combinations) can be associated with diseases that are not easy to detect. For this reason, they want information that reports such cases, including the provenance and date of the information.
- Using this metadata, clinicians want to easily compare all the cases in which a specific phenotypic feature is associated with a disease.

The selection of the right technology does not have a unique answer, but selecting the most suitable one will help you reach the defined goals in a more straightforward way. You can adapt this exercise to different domains and applications.

3.2.2 Representing edge properties with RDF and LPG

From our point of view, LPG is the best solution for representing the data, as it emphasizes information about an edge connecting a phenotypic feature and a disease. To clarify why LPG is the most suitable technology, let's make a concrete comparison between RDF and LPG. The goal is to retrieve all the information (including source, author, and creation date) related to an annotation. As we mentioned earlier, we can use different mechanisms to represent such data using RDF, as described in the following sections.

RDF: N-ARY RELATIONS

A standard approach to modeling data related to a specific edge is to adopt n-ary relations. Using this approach, we create a new concept to connect the data; in our example, this is defined as an annotation. Consider the RDF representation in listing 3.5 and the related SPARQL query in listing 3.6.

Listing 3.5 Example of n-ary relations

```
_:Annotation rdf:type :PhenotypicAnnotation ;
    :forDisease      OMIM:222100 ;
    :phenotypicFeature HP:0410050 ;
    :source          PMID:9357814 ;
    :createdBy       "Nicole Vasilevsky" ;
    :creationDate    "2018-02-23"^^xsd:date .
```

This RDF snippet represents a phenotypic annotation using the Turtle syntax. The annotation is expressed as a blank node (`_:Annotation`), which is an unnamed resource used to group related information without assigning it a global identifier. A blank node can be considered as a placeholder for something that exists but doesn't need a specific name, much like an anonymous object in programming.

The blank node is typed as a `:PhenotypicAnnotation` and links a disease (identified by an OMIM ID) to a phenotypic feature (from the HPO). Additional metadata includes the data source (a PubMed ID), the author of the annotation, and the creation date. This structure supports provenance tracking and semantic interoperability in biomedical datasets.

Listing 3.6 SPARQL query in the context of n-ary relations

```
SELECT ?source ?author ?date
WHERE {
    ?annotation a :PhenotypicAnnotation ;
        :forDisease OMIM:222100 ;
        :phenotypicFeature HP:0410050 ;
        :source ?source ;
        :createdBy ?author ;
        :creationDate ?date .
}
```

This SPARQL query retrieves metadata about a specific phenotypic annotation. It filters annotations by a given disease (OMIM:222100) and phenotypic feature (HP:0410050) and then returns the source of the information, the author who created the annotation, and the date it was created.

In many cases, data consumers can easily interpret and adapt to changes in the original schema. However, as the ontology evolves, its complexity may increase, potentially introducing challenges related to backward compatibility and long-term maintenance.

RDF: NAMED GRAPHS

RDF named graphs include a fourth element specifying that this statement is part of a named (sub)graph and can be considered a node of the RDF graph. Therefore, we can create new statements to attach the data related to the annotation. This approach is represented in listing 3.7, and the SPARQL query is defined in listing 3.8.

Listing 3.7 Example of a named graph

```
:Graph1 {
    OMIM:222100 :hasPhenotypicFeature HP:0410050 .
}

:Graph1
    :source PMID:9357814 ;
    :createdBy "Nicole Vasilevsky" ;
    :creationDate "2018-02-23"^^xsd:date .
```

This RDF example uses TriG syntax to define the named graph `:Graph1`. In simple terms, TriG lets you group RDF statements under a label (the named graph) and add metadata. In this graph, the triple asserts that the disease OMIM:222100 has the phenotypic feature HP:0410050. Metadata about this assertion is attached to `:Graph1`, including the source (PMID:9357814), the creator ("Nicole Vasilevsky"), and the creation date.

Listing 3.8 SPARQL query in the context of a named graph

```
SELECT ?source ?author ?date
WHERE {
    GRAPH :Graph1 {
        OMIM:222100 :hasPhenotypicFeature HP:0410050 .
    }
    :Graph1 :source ?source ;
           :createdBy ?author ;
           :creationDate ?date .
}
```

This SPARQL query retrieves metadata about a specific phenotypic annotation stored in a named graph. It looks in graph `:Graph1` to find a triple asserting that

OMIM:222100 has the phenotypic feature HP:0410050. It then queries the metadata about :Graph1 and returns the source, the author, and the creation date.

Although named graphs are powerful for representing contextual metadata and provenance, they can add complexity. In particular, managing a large number of named graphs may lead to inefficiencies in data storage and exchange. Fine-grained updates to individual statements in named graphs can also be challenging.

RDF-STAR

As previously mentioned, RDF-star is an extension of RDF that narrows the gap between RDF and property graph models such as LPG. This approach is illustrated in the following two listings.

Listing 3.9 Example of RDF-star

```
<<OMIM:222100 :hasPhenotypicFeature HP:0410050>>
  :source PMID: 9357814 ;
  :createdBy "Nicole Vasilevsky" ;
  :creationDate "2018-02-23"^^xsd:date .
```

Listing 3.10 Example of a SPARQL-star query in the context of RDF-star

```
SELECT ?source ?author ?date {
  <<OMIM:222100 :hasPhenotypicFeature HP:0410050>>
    :source ?source ;
    :createdBy ?author ;
    :creationDate ? date .
}
```

RDF-star represents a step in attaching properties to edges and uses a more readable SPARQL query. However, its query performance must be improved; and, as noticed by Orlandi et al. [2], “The use of a new syntax extension requires a specific implementation of RDF engines and, therefore, limits the adoption of this approach.”

Other methods exist for annotating RDF statements, such as reification and singleton properties. These methods are less used in real-world applications, where more scalable and maintainable alternatives like named graphs and n-ary relations are preferred.

LPG

The LPG approach represents annotation details directly within the relationship, using key–value pairs. An example of this modeling approach and the corresponding Cypher query are shown next.

Listing 3.11 Example LPG representation

```
(d { id: "OMIM:222100" })
-[:HAS_PHENOTYPIC_FEATURE {
  source: "PMID:9357814"
```

```

        createdBy: "Nicole Vasilevsky";
        creationDate: "2018-02-23"]->
(p { id: "HP:0410050" })

```

The two nodes represent entities: a disease (OMIM:222100) and a phenotype (HP:0410050). The relationship `:HAS_PHENOTYPIC_FEATURE` connects them and includes key-value pairs that describe the source of the annotation ("PMID:9357814"), the creator ("Nicole Vasilevsky"), and the date it was created ("2018-02-23").

Listing 3.12 Example Cypher query

```

MATCH (d)-[r:HAS_PHENOTYPIC_FEATURE]->(p)
WHERE d.id = "OMIM:222100" and p.id = "HP:0410050"
RETURN r.source, r.createdBy, r.creationDate

```

This Cypher query retrieves the metadata attached to the `:HAS_PHENOTYPIC_FEATURE` relationship between the disease and phenotype nodes. It matches the pattern in the graph, filters based on the node IDs, and returns the annotation details stored in the relationship.

As these examples demonstrate, the LPG model is well-suited for modeling metadata-rich relationships in a way that is expressive and accessible. For these reasons, we'll adopt LPG and Cypher as the core tools for building our KG system.

3.3 *Building a knowledge graph*

Let's get into the details of how to build our first KG. The process has two steps: loading the ontology and ingesting a data source using the ontology as a reference.

NOTE To build the KG, you can run the code in the GitHub repository (<https://github.com/alenegro81/knowledge-graphs-and-llms-in-action/tree/main/chapters/ch03>) or test the Cypher queries in this section using the Neo4j browser. The code has been tested using Neo4j (version 5.20.0 Enterprise Edition, installed with the Neo4j Desktop 1.6.1 application), the APOC library (version 5.20.0), and the Neosemantics plugin (version 5.20.0). Details for installing Neo4j and its plugins are provided in online appendix B. We explain each query, but we assume you have a basic understanding of the Cypher query language. The results are derived from the HPO version available in February 2025.

3.3.1 *Ontology ingestion and processing with neosemantics*

Figure 3.7 illustrates the ontology ingestion and processing phase. The first step is to create and initialize the HPO database using the following command.

Listing 3.13 Creating the HPO database in Neo4j

```
CREATE DATABASE hpo IF NOT EXISTS
```

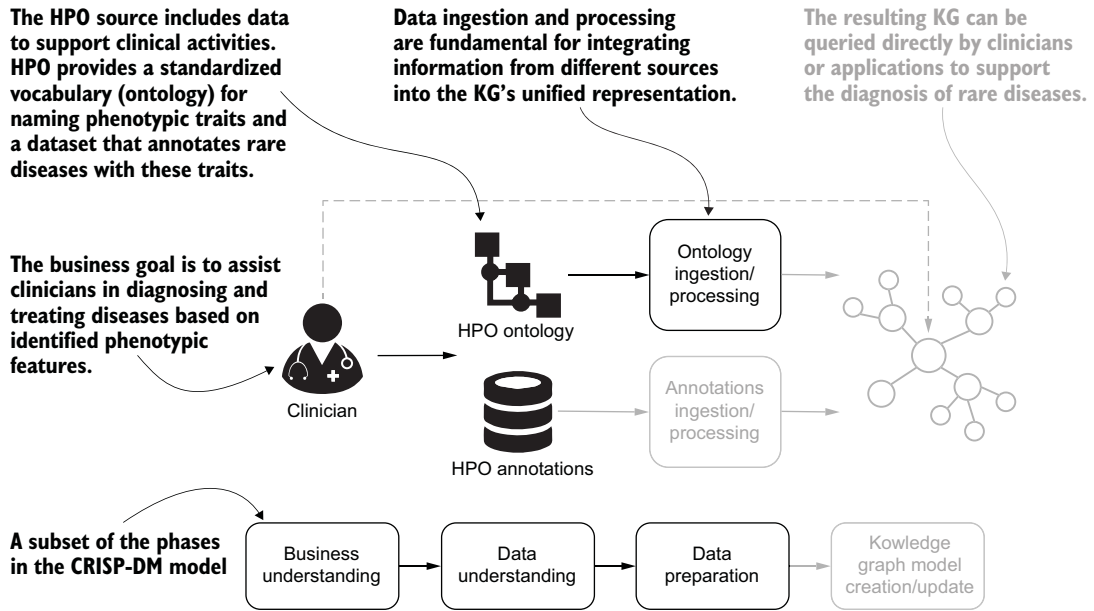


Figure 3.7 Ontology ingestion and processing

In the next listing, we establish constraints that ensure the uniqueness of the `uri` and `id` properties of the nodes labeled `Resource`. We also create indexes for the `id` properties of `HpoPhenotype` and `HpoDisease` nodes to enhance access to this information during the KG building phase and information retrieval. The `HpoPhenotype` and `HpoDisease` labels define our phenotypic abnormality and disease nodes.

Listing 3.14 Creating constraints and indexes

```
CREATE CONSTRAINT n10s_unique_uri IF NOT EXISTS FOR (r:Resource) REQUIRE
    r.uri IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (n:Resource) REQUIRE (n.id) IS UNIQUE;
CREATE INDEX disease_id IF NOT EXISTS FOR (n:HpoDisease) ON (n.id);
CREATE INDEX phenotype_id IF NOT EXISTS FOR (n:HpoPhenotype) ON (n.id);
```

The second step defines an initial configuration for the Neosemantics component.

Listing 3.15 Configuring the Neosemantics plugin

```
CALL n10s.graphconfig.init();
CALL n10s.graphconfig.set({ handleVocabUris: "IGNORE" });
CALL n10s.graphconfig.set({ applyNeo4jNaming: True });
```

This configuration defines two main rules for importing data. The first rule ignores the namespaces in the import phase (namespaces can help keep track of distinct

ontologies that use similar expressions.) The second rule encodes the relationship types in uppercase, following the standard representation of LPG relationships.

The next step is to load the HPO vocabulary.

Listing 3.16 Loading the HPO vocabulary into Neo4j

```
CALL n10s.rdf.import.fetch("http://purl.obolibrary.org/obo/hp.owl", "RDF/XML");
```

During our tests, this command loaded 899,558 statements into Neo4j. Before processing and loading the annotation data, we can enrich our nodes with the `Hpo-Phenotype` label and the `id` property computed from the resource's original URI.

Listing 3.17 Enriching nodes

```
MATCH (n:Resource)
WHERE n.uri STARTS WITH "http://purl.obolibrary.org/obo/HP"
SET n:HpoPhenotype,
    n.id = coalesce(n.id,
        replace(apoc.text.replace(n.uri, '(.*)obo/', ''), '_', ':'))
```

Sets n.id as
HP:0000001

Let's review the current state of the KG. Listing 3.18 shows the code to retrieve a small portion of this graph, illustrated in figure 3.8. You can explore this by running the code in the Neo4j browser.

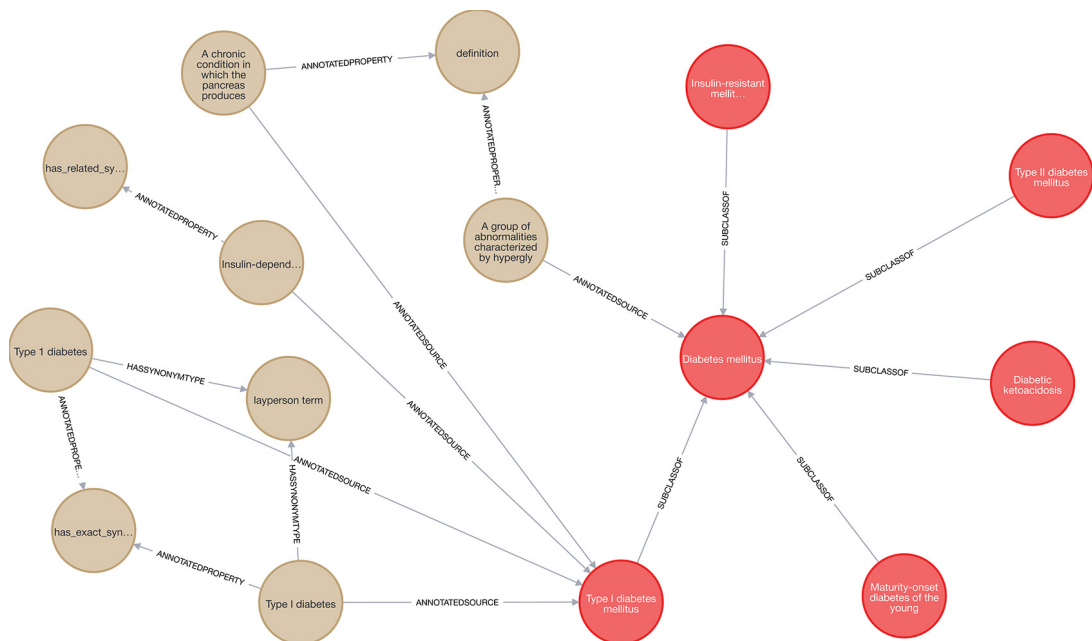


Figure 3.8 A portion of the HPO ontology loaded in the graph database using LPG as a storage model. We can distinguish between two types of information: ontological information (left) and domain-specific information related to phenotypic features (right).

Listing 3.18 Showing part of the KG at the current stage

```

MATCH path1=(n:HpoPhenotype)<-[:SUBCLASSOF]-(m:HpoPhenotype)
WHERE n.label = "Diabetes mellitus"
WITH path1
MATCH path2=(i:HpoPhenotype)<-[:ANNOTATEDSOURCE]-(j)
WHERE i.label in ["Diabetes mellitus", "Type I diabetes mellitus"]
WITH path1, path2, j
MATCH path3=(j)-[:ANNOTATEDPROPERTY|HAS_SYNONYM_TYPE]-()
RETURN path1, path2, path3

```

WARNING The query in listing 3.18 will work only if you execute it while following the chapter instructions one step at a time. If you run the entire ingestion process using the repository code, the query will fail due to the final data cleaning phase.

The HPO ontology provides different types of information. The left side of figure 3.8 reports ontological information about the nature of the nodes, and the right side includes details on hierarchical connections related to diabetes mellitus.

3.3.2 Annotation ingestion and processing

To finish constructing the KG, we must ingest and process the annotations file. The phenotypic abnormalities in this file are connected to the associated diseases, whose terms come from other ontologies. Figure 3.9 shows the second phase of the data processing and modeling.

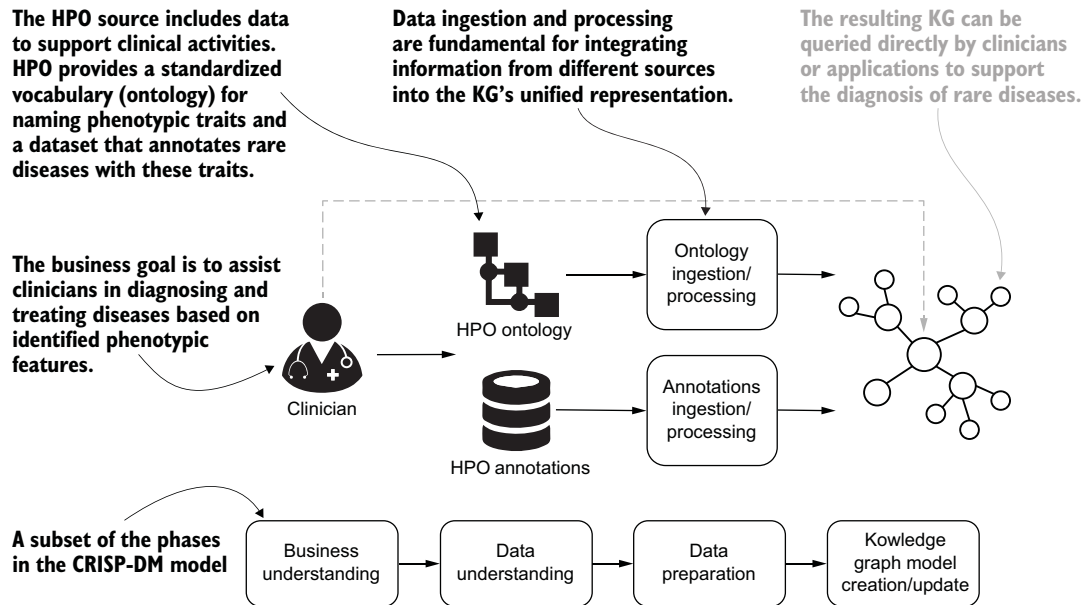


Figure 3.9 Ingesting and processing the annotation dataset to finalize the construction of the KG

Unlike the `hpo.owl` file generated using the RDF data model, our next file is provided in an HPO annotation (HPOA; <https://mng.bz/NwQN>) format that consists of tab-separated-values (TSVs). The HPOA file includes valuable information:

- An explicit association between a disease and multiple phenotypic features or abnormalities
- Evidence supporting this association, such as that it's inferred from an electronic annotation or from a published clinical study or traceable author statement
- The age of onset
- The frequency with which a disease and a phenotypic feature appear together
- Additional metadata that describes the ontology source

Working with this TSV file allows us to incorporate different file types based on existing knowledge. The Cypher queries in listings 3.19–3.24 let us load, process, and integrate information from the annotations file on GitHub. First we create disease nodes.

Listing 3.19 Creating HpoDisease nodes

```
LOAD CSV FROM 'https://mng.bz/qRyr' AS row
FIELDTERMINATOR '\t'
WITH row
SKIP 5
MERGE (dis:Resource:HpoDisease {id: row[0]})
ON CREATE SET dis.label = row[1];
```

← Skips the first five rows of the file because they are file metadata

Next we create the relationships between disease nodes and phenotypic feature nodes.

Listing 3.20 Creating relationships between HpoDisease and HpoPhenotype nodes

```
LOAD CSV FROM 'https://mng.bz/qRyr' AS row
FIELDTERMINATOR '\t'
WITH row
SKIP 5
MATCH (dis:HpoDisease)
WHERE dis.id = row[0]
MATCH (phe:HpoPhenotype)
WHERE phe.id = row[3]
MERGE (dis)-[:HAS_PHENOTYPIC_FEATURE]->(phe)
```

Creating these relationships integrates information from the `hpo.owl` and `phenotype.hpoa` files. The following code queries the result of this integration process.

Listing 3.21 Finding associations

```
MERGE (dis:HpoDisease)-[:HAS_PHENOTYPIC_FEATURE]->(phe:HpoPhenotype)
RETURN dis.label, collect(phe.label)
LIMIT 3
```

The results of the query are reported in table 3.1.

Table 3.1 Sample associations between the HpoDisease and HpoPhenotype nodes

HpoDisease entry	Associated HpoPhenotype entries
Developmental and epileptic encephalopathy 96	Hydrops fetalis, Autosomal dominant inheritance, Death in infancy, Epileptic spasm, Primary microcephaly, EEG with burst suppression, Intellectual disability, profound, Small for gestational age, Epileptic encephalopathy, Neonatal respiratory distress, Tonic seizure
Pseudohyperkalemia, familial, 2, due to red cell leak	Generalized muscle weakness, Hyperkalemia, Periodic paralysis, Muscle spasm, Hemolytic anemia, Hand tremor, Autosomal dominant inheritance
Immunoglobulin kappa light chain deficiency	Chronic diarrhea, Recurrent infections, Recurrent respiratory infections, Absent circulating immunoglobulin kappa chain, Childhood onset, Diarrhea, Autosomal recessive inheritance

The following code adds relationship properties in the form of key–value pairs.

Listing 3.22 Adding properties to HAS_PHENOTYPIC_FEATURE relationships

```
LOAD CSV FROM 'https://mng.bz/qRyr' AS row
FIELDTERMINATOR '\t'
WITH row
SKIP 5
MATCH (dis:HpoDisease) - [rel:HAS_PHENOTYPIC_FEATURE] -> (phe:HpoPhenotype)
WHERE phe.id = row[3] and dis.id = row[0]
FOREACH(_ IN CASE WHEN row[4] is not null THEN [1] ELSE [] END |
  SET rel.source = row[4])
FOREACH(_ IN CASE WHEN row[5] is not null THEN [1] ELSE [] END |
  SET rel.evidence = row[5])
FOREACH(_ IN CASE WHEN row[6] is not null THEN [1] ELSE [] END |
  SET rel.onset = row[6])
FOREACH(_ IN CASE WHEN row[7] is not null THEN [1] ELSE [] END |
  SET rel.frequency = row[7])
FOREACH(_ IN CASE WHEN row[8] is not null THEN [1] ELSE [] END |
  SET rel.sex = row[8])
FOREACH(_ IN CASE WHEN row[9] is not null THEN [1] ELSE [] END |
  SET rel.modifier = row[9])
FOREACH(_ IN CASE WHEN row[10] is not null THEN [1] ELSE [] END |
  SET rel.aspect = row[10])
FOREACH(_ IN CASE WHEN row[11] is not null THEN [1] ELSE [] END |
  SET rel.biocuration = row[11])
```

This is a flexible approach to enrich relationship information. This script matches existing nodes and relationships in the Neo4j graph and sets additional relationship properties based on the presence of values in each row of the input file. Each of the FOREACH blocks adds a new property to the relationship only if the corresponding column in the TSV is not null. This makes the script resilient to missing data and avoids overwriting values with nulls.

Next we incorporate the information from the following query to clarify the meaning of the properties associated with the relationships between diseases and phenotypic features.

Listing 3.23 Enriching HAS_PHENOTYPIC_FEATURE with more properties

```
CALL apoc.periodic.iterate(
  "MATCH (dis:HpoDisease)-[rel:HAS_PHENOTYPIC_FEATURE]->(phe:HpoPhenotype)
  RETURN rel",
  "SET rel.createdBy = apoc.text.regexGroups(
    rel.biocuration, 'HPO:(\\w+)\\['
  )[0][1],
  rel.creationDate = apoc.text.regexGroups(
    rel.biocuration, '\\[(\\d{4})-(\\d{2})-(\\d{2})\\]'
  )[0][1],
  rel.aspectName = CASE
    WHEN rel.aspect = 'P' THEN 'Phenotypic abnormality'
    WHEN rel.aspect = 'I' THEN 'Inheritance'
  END,
  rel.aspectDescription = CASE
    WHEN rel.aspect = 'P' THEN
      'Terms with the P aspect are located in the Phenotypic abnormality ' +
      'subontology'
    WHEN rel.aspect = 'I' THEN
      'Terms with the I aspect are from the Inheritance subontology'
  END,
  rel.evidenceName = CASE
    WHEN rel.evidence = 'IEA' THEN
      'Inferred from electronic annotation'
    WHEN rel.evidence = 'PCS' THEN
      'Published clinical study'
    WHEN rel.evidence = 'TAS' THEN
      'Traceable author statement'
  END,
  rel.evidenceDescription = CASE
    WHEN rel.evidence = 'IEA' THEN
      'Annotations extracted by parsing the Clinical Features sections ' +
      'of the Online Mendelian Inheritance in Man resource are assigned ' +
      'the evidence code IEA.'
    WHEN rel.evidence = 'PCS' THEN
      'PCS is used for information extracted from articles in the medical ' +
      'literature. Generally, annotations of this type will include the ' +
      'pubmed id of the published study in the DB_Reference field.'
    WHEN rel.evidence = 'TAS' THEN
      'TAS is used for information gleaned from knowledge bases such as ' +
      'OMIM or Orphanet that have derived the information from a ' +
      'published source.'
  END,
  rel.url = CASE
    WHEN rel.source STARTS WITH 'PMID:' THEN
      'https://pubmed.ncbi.nlm.nih.gov/' + apoc.text.replace(
        rel.source, '(.*)PMID:', ''
      )
  )
)
```

```

    WHEN rel.source STARTS WITH 'OMIM:' THEN
      'https://omim.org/entry/' + apoc.text.replace(
        rel.source, '(.*)OMIM:', ''
      )
    END",
    {batchSize: 1000}
  )
)

```

This query uses `apoc.periodic.iterate` to process and update the `HAS_PHENOTYPIC_FEATURE` relationships in batches. For example, it creates metadata from the biocuration property by extracting the curator and creation date using a regular expression. The query also adds properties to improve readability during graph exploration. The annotation file includes abbreviated versions of information related to aspect (P or I values) and evidence (IEA, PCS, or TAS values). To clarify this data, we add properties such as `aspectName`, which can have the value 'Phenotypic abnormality' or 'Inheritance'. The goal is to make it easier for humans to access information.

The final step in building the KG is to clean it by removing nodes and relationships that come from the ontology but are not necessary for our purposes.

Listing 3.24 Cleaning the KG by removing unnecessary nodes and relationships

```

CALL apoc.periodic.iterate(
  "MATCH (n:Resource) RETURN id(n) as id",
  "MATCH (n)
   WHERE id(n) = id AND
         NOT 'HpoPhenotype' in labels(n) AND
         NOT 'HpoDisease' in labels(n)
   DETACH DELETE n",
  {batchSize:10000})
YIELD batches, total return batches, total

```

3.4 Querying the data

Clinicians can now use the KG as a support tool for diagnosing rare diseases, beginning with the detection of phenotypic abnormalities in a patient. By entering specific traits, clinicians can query the KG to identify rare pathologies. This querying phase is the final step of our mental model and is shown in figure 3.10.

Imagine that a clinician sees a patient: a boy affected by Type 1 diabetes. The patient's clinical history is stored in the hospital database as an electronic health record (EHR). The hospital has embraced the KG paradigm change, so patient information is stored using the terms included in HPO and OMIM (an online catalog of genetic disorders and rare diseases). Type 1 diabetes is classified as a phenotypic feature and a disease, so the information is stored using two different identification codes:

- HP:0100651 (phenotypic feature): <https://hpo.jax.org/app/browse/term/HP:0100651>.
- OMIM:222100 (disease): <https://www.omim.org/entry/222100>.

The HPO source includes data to support clinical activities. HPO provides a standardized vocabulary (ontology) for naming phenotypic traits and a dataset that annotates rare diseases with these traits.

Data ingestion and processing are fundamental for integrating information from different sources into the KG's unified representation.

The resulting KG can be queried directly by clinicians or applications to support the diagnosis of rare diseases.

The business goal is to assist clinicians in diagnosing and treating diseases based on identified phenotypic features.

A subset of the phases in the CRISP-DM model

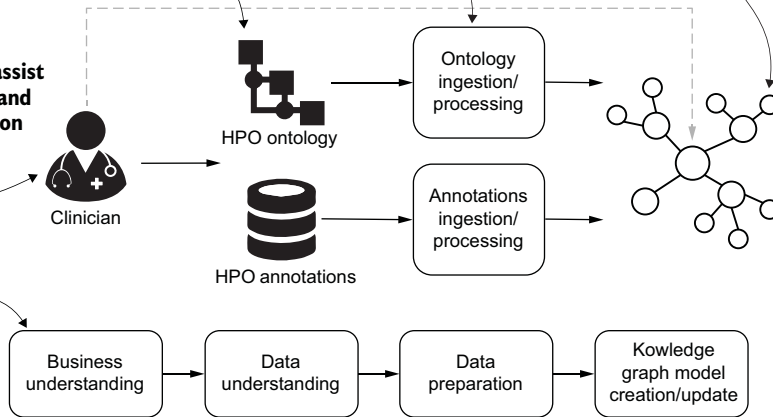


Figure 3.10 Querying the generated KG to support clinicians' activities

The clinician recognizes the typical phenotypic features of Type 1 diabetes in the patient, which can also be explored in the KG with the query in the following listing. Figure 3.11 shows the results.



Figure 3.11 Result of a query that gets all the phenotype features related to Type 1 diabetes

Listing 3.25 Querying phenotypic features associated with Type 1 diabetes

```
MATCH path=(dis:HpoDisease)-[:HAS_PHENOTYPIC_FEATURE]->(phe:HpoPhenotype)
WHERE dis.id = "OMIM:222100"
RETURN path
```

The central node defines Type 1 diabetes, and the other nodes define the associated phenotypic features. However, during the medical examination, the clinician recognizes new symptoms classified as phenotypic features that are not directly connected to Type 1 diabetes:

- Growth delay: <https://hpo.jax.org/app/browse/term/HP:0001510>.
- Large knee: <https://hpo.jax.org/app/browse/term/HP:0030866>.
- Sensorineural hearing impairment: <https://hpo.jax.org/app/browse/term/HP:0000407>.
- Pruritus: <https://hpo.jax.org/app/browse/term/HP:0000989>.

The clinician wants to use the information in the KG to identify other pathologies connected to these phenotype features. To perform this task, the clinician runs the following query, which gives the results listed in table 3.2.

Listing 3.26 Finding diseases associated with specific phenotypic features

```
MATCH (phe:HpoPhenotype)
WHERE phe.label IN [
    "Growth delay",
    "Large knee",
    "Sensorineural hearing impairment",
    "Pruritus",
    "Type I diabetes mellitus"
]
WITH phe
MATCH path=(dis:HpoDisease)-[:HAS_PHENOTYPIC_FEATURE]->(phe)
UNWIND dis as nodes
RETURN
    dis.id as disease_id,
    dis.label as disease_name,
    collect(phe.label) as features,
    count(nodes) as num_of_features
ORDER BY num_of_features DESC, disease_name
LIMIT 5
```

Table 3.2 Top diseases matching clinician-identified phenotypic features

disease_id	disease_name	features	num_of_features
OMIM:619269	Ondotochondrodysplasia 2 with hearing loss and diabetes	Growth delay, Sensorineural hearing impairment, Pruritus, Large knee, Type I diabetes mellitus	5
OMIM:618500	Holoprosencephaly 12 with or without pancreatic agenesis	Sensorineural hearing impairment, Growth delay, Type I diabetes mellitus	3

Table 3.2 Top diseases matching clinician-identified phenotypic features (*continued*)

disease_id	disease_name	features	num_of_features
OMIM:614700	3-methylglutaconic aciduria, type VIII	Growth delay, Sensorineural hearing impairment	2
OMIM:616192	Alobar holoprosencephaly	Growth delay, Sensorineural hearing impairment	2
OMIM:602782	Alpha-Thalassemia/mental retardation syndrome, X-linked	Growth delay, Sensorineural hearing impairment	2

These results lead to a diagnosis of Ondontochoondrodysplasia 2 with hearing loss and diabetes. Starting with these results, the clinician can conduct further investigations to determine the frequency with which these phenotypic features are associated with the disease and identify more potential sources of information.

Exercise

Extend the query in listing 3.26 to retrieve relationship properties including `evidence_name`, `evidence_description`, `source`, and `url`.

3.5 Reasoning over the KG

In the previous case, we showed how to obtain results from the information stored in the KG. However, one of the most powerful tools of a KG is inference, which uses deductive reasoning (see chapter 2) based on logical rules to derive results from implicit information. For example, consider this question: which diseases are characterized by an abnormality of the endocrine system?

Some annotations are explicitly connected to this phenotypic feature. But a clinician would also be interested in more specific phenotypic traits that involve the thyroid. For this purpose, we can use the hierarchical representation of HPO. The following query retrieves a subset of phenotypic features representing subclasses of endocrine system abnormalities (`id=HP:0000818`).

Listing 3.27 Finding subclasses of endocrine system abnormalities

```
MATCH (p:HpoPhenotype) <- [:SUBCLASSOF*1..3] - (n:HpoPhenotype)
WHERE p.id = "HP:0000818"
RETURN p,n
```

Finds all phenotype nodes (n) that are one to three subclass levels more specific than another phenotype node (p)

Using this hierarchical structure, we can infer annotations implicitly linked to the abnormalities of the endocrine system through the following Neosemantics procedure (listing 3.28). Table 3.3 shows a subset of the results.

Listing 3.28 Finding phenotypic features related to the abnormality subclasses

```

MATCH (cat:HpoPhenotype {label: "Abnormality of the endocrine system"})
CALL n10s.inference.nodesInCategory(cat, {
  inCatRel: "HAS_PHENOTYPIC_FEATURE",
  subCatRel: "SUBCLASSOF"})
YIELD node as dis
WHERE dis.label IN [
  "Congenital atransferrinemia",
  "Deafness, autosomal recessive 4, with enlarged vestibular aqueduct",
  "Diabetes mellitus, transient neonatal, 1",
  "Edema, familial idiopathic, prepubertal",
  "Familial dysalbuminemic hyperthyroxinemia"
]
MATCH (dis) - [:HAS_PHENOTYPIC_FEATURE] -> (phe:HpoPhenotype)
RETURN dis.label as disease, collect(DISTINCT phe.label) as features
ORDER BY size(features) ASC, disease

```

← Finds the top-level phenotype node

← Gets diseases linked (directly or indirectly) to this phenotype

← Keeps only selected diseases for reproducible output

← Matches their phenotype features

Table 3.3 Subset of the results of annotations implicitly connected to the “Abnormality of the endocrine system” phenotypic feature. Phenotypic features that are direct or inferred subclasses of this phenotypic feature are highlighted in bold.

disease	features
Congenital atransferrinemia	Anemia, Abnormality of the pancreas, Recurrent infections, Arthritis, Abnormality of the cardiovascular system, Hypothyroidism
Deafness, autosomal recessive 4, with enlarged vestibular aqueduct	Enlarged vestibular aqueduct, Congenital onset, Goiter , Autosomal recessive inheritance, Incomplete partition of the cochlea type II, Sensorineural hearing impairment
Diabetes mellitus, transient neonatal, 1	Transient neonatal diabetes mellitus , Autosomal dominant inheritance, Dehydration, Hyperglycemia, Intrauterine growth retardation, Severe failure to thrive
Edema, familial idiopathic, prepubertal	Diabetes mellitus , Abnormality of the genitourinary system, Irritability, Vomiting, Autosomal dominant inheritance, Edema
Familial dysalbuminemic hyperthyroxinemia	Abnormal circulating free T4 concentration, Abnormal thyroid-stimulating hormone level , Autosomal dominant inheritance, Autosomal recessive inheritance, Euthyroid hyperthyroxinemia, Increased circulating free T4 concentration

These results demonstrate how reasoning over subclass relationships and phenotypic features can reveal meaningful disease associations within an ontology-driven graph. The use of the Neosemantics plugin highlights the power of semantic inference in enriching biomedical queries, enabling us to go beyond direct connections and tap into the structure of domain knowledge.

Summary

- KG construction is a complex process that requires a clear idea of the problem you want to solve, an understanding of the reference domain, and a phase that includes data scouting, exploration, and comprehension.
- The resulting KG must be a unified, well-grounded, meaningful representation of data from different sources, with individual pieces of information fused into a unique view.
- The Resource Description Framework (RDF) and Labeled Property Graph (LPG) are two of the most prominent technologies for building KGs.
 - The RDF data model focuses on knowledge representation and is particularly suitable for constructing ontologies.
 - The LPG approach provides fast, query-based traversal of graph data and path analysis, emphasizing the efficiency of data storage and access.
 - Understanding the differences between RDF and LPGs is crucial for selecting the best technologies for your specific purpose.